

F4 Aktivitetsdiagram

DA343A, Institutionen för datavetenskap och medieteknik

Innehåll

- Repetition
- Aktivitetsdiagram och kod
- Mer komplexa diagram
- Notation och demo i Visual Paradigm
- Sammanfattning



Repetition

Repetition

- Ni har sett aktivitetsdiagram i kursen DA339A
- Beskriver en *aktivitet* i stegform
- Kan visa olika scenarion

Varför vill vi ha aktivitetsdiagram?

- Diagramet *kommunicerar* en lösning som är oberoende av kodspråk
 - Programmeringsspråk har olika syntax
 - Antal Java-rader för att skriva ut "Hej": 3
 - Antal JavaScript-rader för att skriva ut "Hej": 1
- Det kan förbereda för att skriva en lösning i kod
- Kan liknas vid "grafisk" pseudokod
- Olika abstraktions-nivåer låter dig beskriva samma lösning på flera sätt

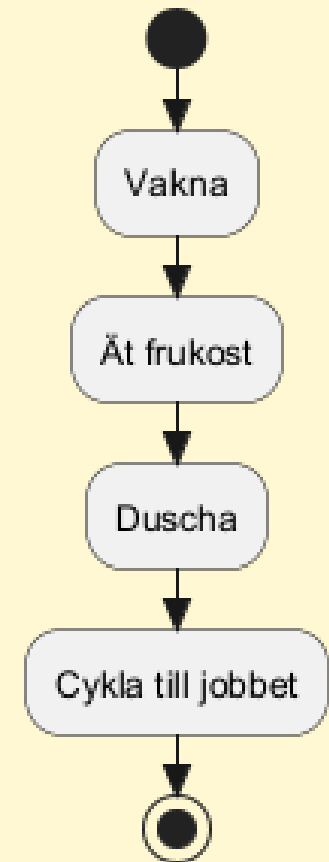
Ett enkelt exempel: Min morgon

I text:

På morgonen när väckarklockan ringer så vaknar jag, går upp och äter frukost. Efter det tar jag en dusch innan jag cyklar till jobbet och blir svettig på nytt.

I punktform:

- Jag vaknar
- Äter frukost
- Duschar
- Cyklar till jobbet



Viktiga saker att komma ihåg

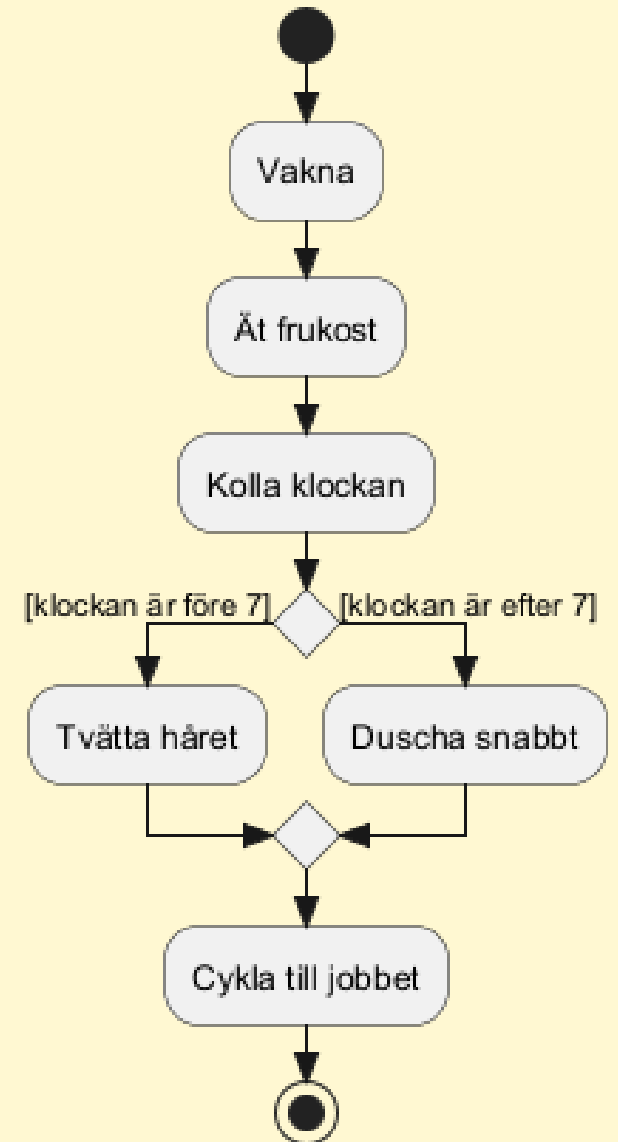
- Varje handling (*eng: action*) har minst två pilar kopplade till sig
 - Minst en som pekar in
 - Exakt en som pekar ut
 - Pilar kallas även ”bågar” (*eng: edges*)
- Det får inte finnas ”återvändsgränder”
- Handlingar ska skrivas koncist och beskrivande
 - Något av en motsägelse

Min morgon, med villkor

Att tvätta håret tar tid (och borde inte göras varje dag). Nu börjar det bli besvärligare att beskriva i textform eller punktform.

På morgonen när väckar-klockan ringer, går jag upp och äter frukost. Efter det tar jag en dusch, och om klockan är före 7 tvättar jag håret. Sen cyklar jag till jobbet och blir svettig på nytt

- Jag vaknar
- Äter frukost
- Om klockan före 7
 - Tvättar håret
- Klockan efter 7
 - Duschar snabbt
- Cyklar till jobbet

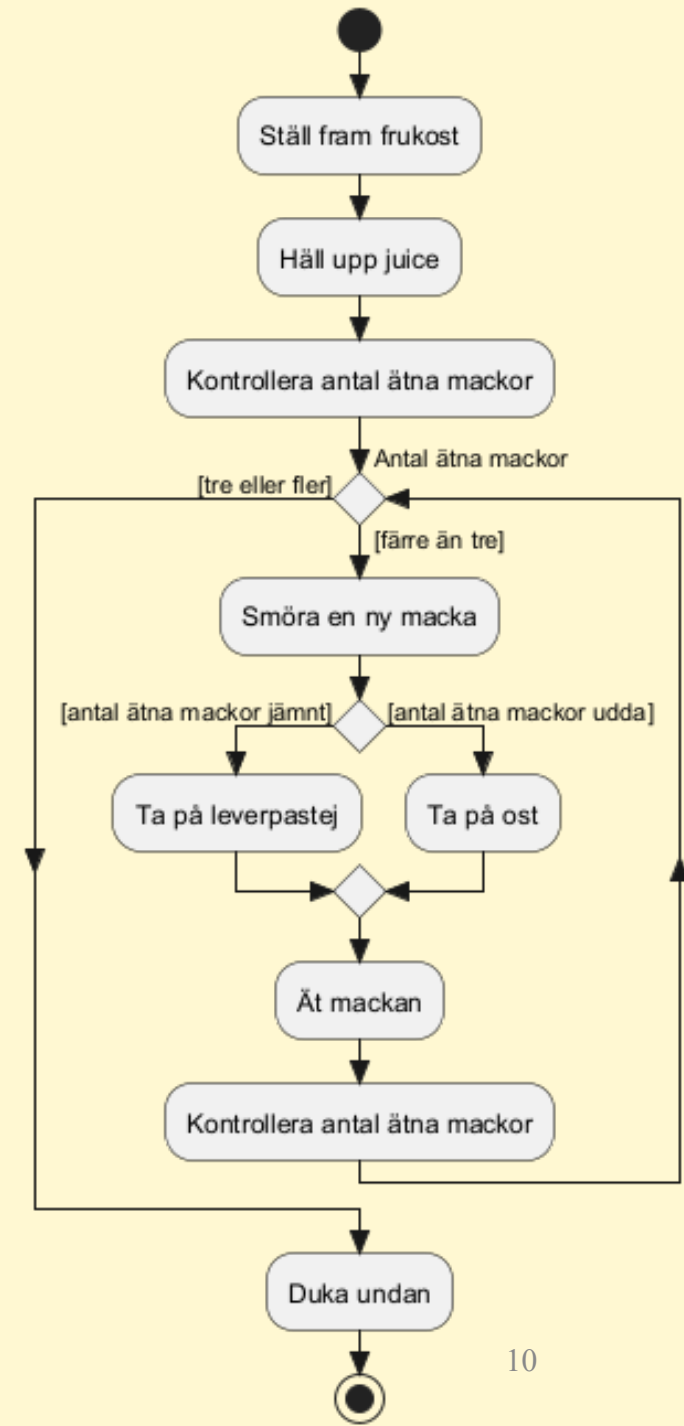


Kommentar om komplexitet

- I de tidigare exemplet var det fortfarande möjligt att tydligt beskriva hur morgonen går till i text- och punktform
- Men allt eftersom vi lägger till steg och villkor blir det svårare
- Vi kommer att komma till parallellisering senare och då blir det ännu klurigare att tydligt förmedla i vilken ordning saker sker i textform.

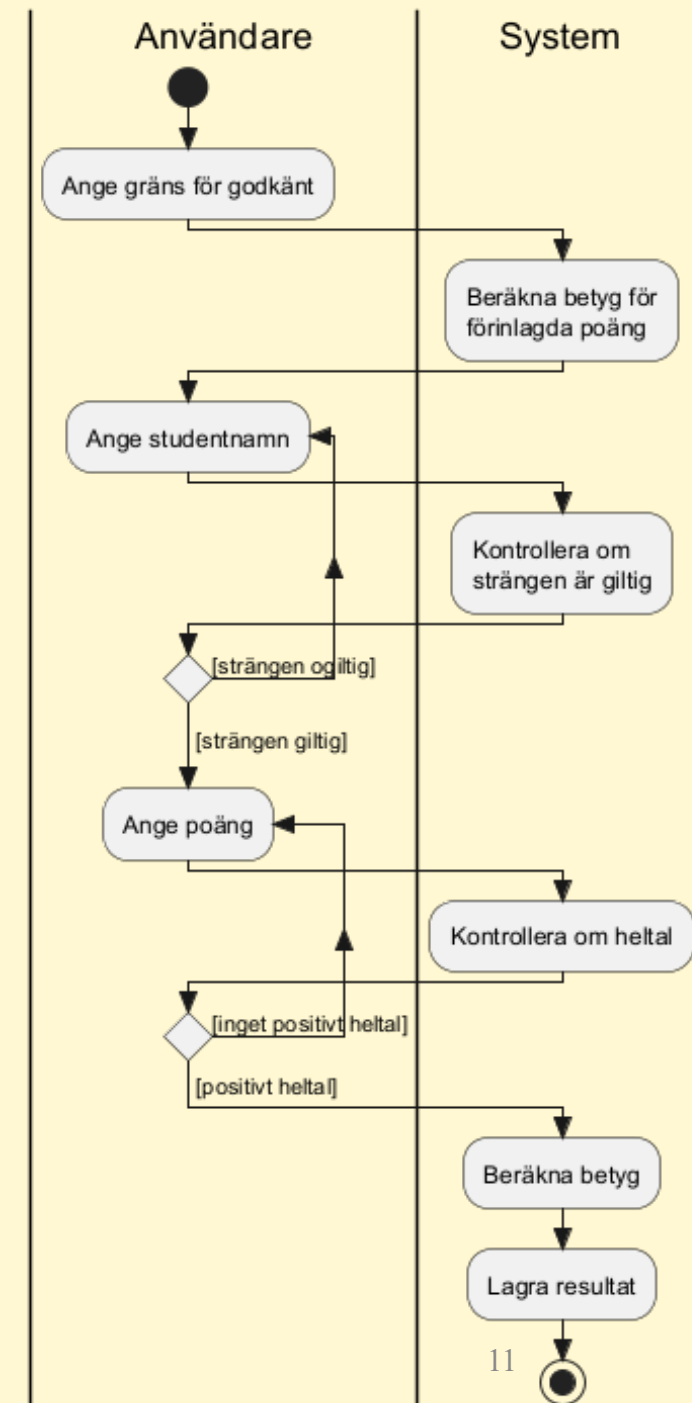
Min frukost, med en loop

- Vi kan ”zooma” in på enskilda delar i ett aktivitetsdiagram
- En loop markeras inte på något särskilt sätt, mer än att vi skapar en slinga.
- Detaljnivån man vill ha beror på sammanhanget.
 - Ibland på koncepts-nivå
 - Ibland på metodanrops-nivå
 - Ibland på kodrads-nivå
 - Kolla uppgiftsbeskrivningen



Swimlanes

- Ibland vill man visa vem som utför vilken handling
- Detta visualiseras med hjälp av *swimlanes*
- Till höger visas ett scenario där man försöker lägga till provresultat
- (Här kan jag tycka att romberna borde ligga i den högra swimlane:n, men mitt UML-program låter mig inte styra över placeringar)
- Swimlanes kan vara både vertikala och horisontella

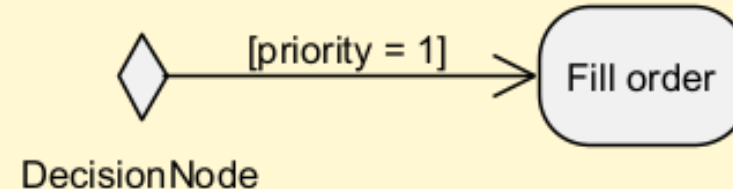
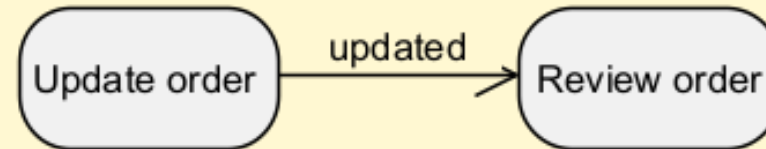
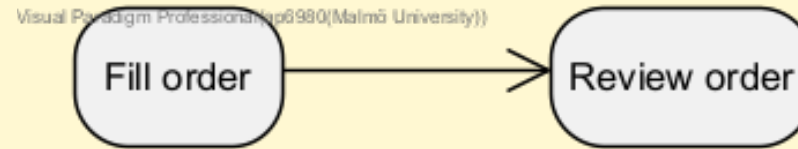


Aktivitet vs. handling

- Engelska: *activity* vs. *action*
- En *activity* (aktivitet) består av *actions* (handlingar)
- Ser likadana ut, båda är rutor med rundade hörn
- En *action* kan inte brytas ner mer (i den abstraktionsnivån man är på)
- En *action* kan vara en *activity* i en annan abstraktionsnivå
 - Vi kan se ”äta lunch” som en *action* om vi kollar på vad vi gör under en dag
 - Men att äta lunch består av flera handlingar (*actions*) och är på så vis en aktivitet (*activity*)

Bågar (edges)

- Pilarna mellan handlingar kallas för bågar (*eng: edges*)
- Bågarna har en riktning (markeras som en pil)
- Bågarna kan annoteras
- Villkor kallas *guards* och symboliseras med hakparenteser

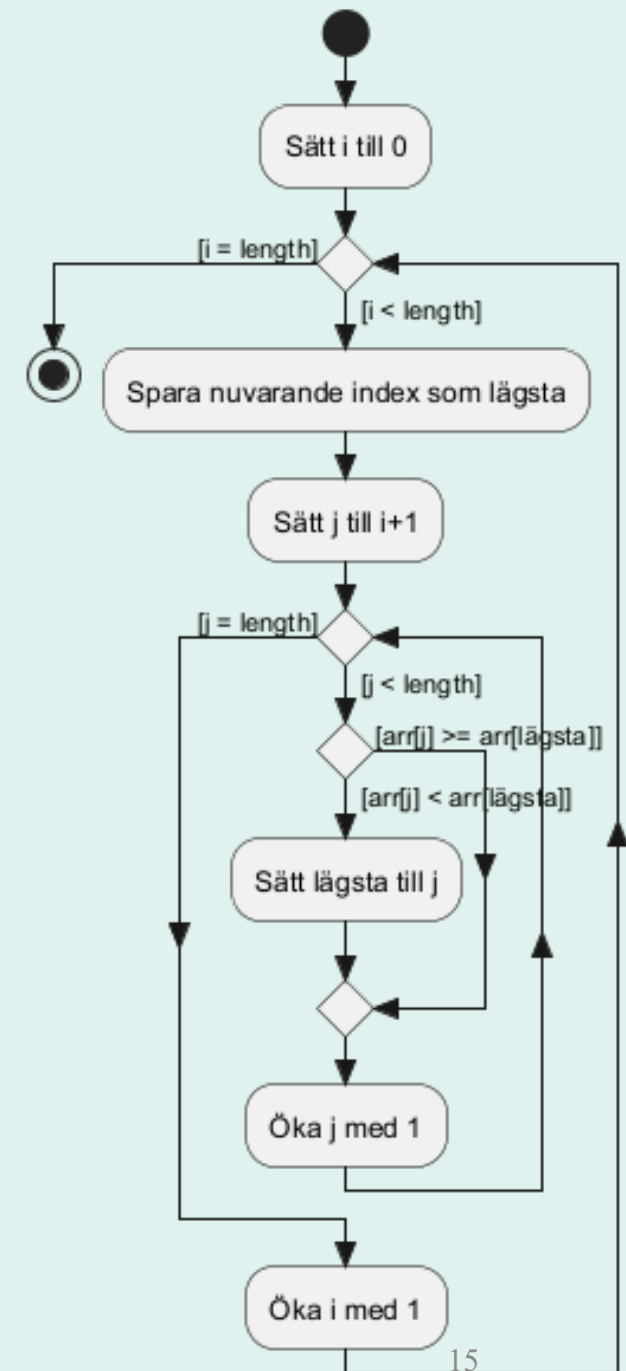




Aktivitetsdiagram och kod

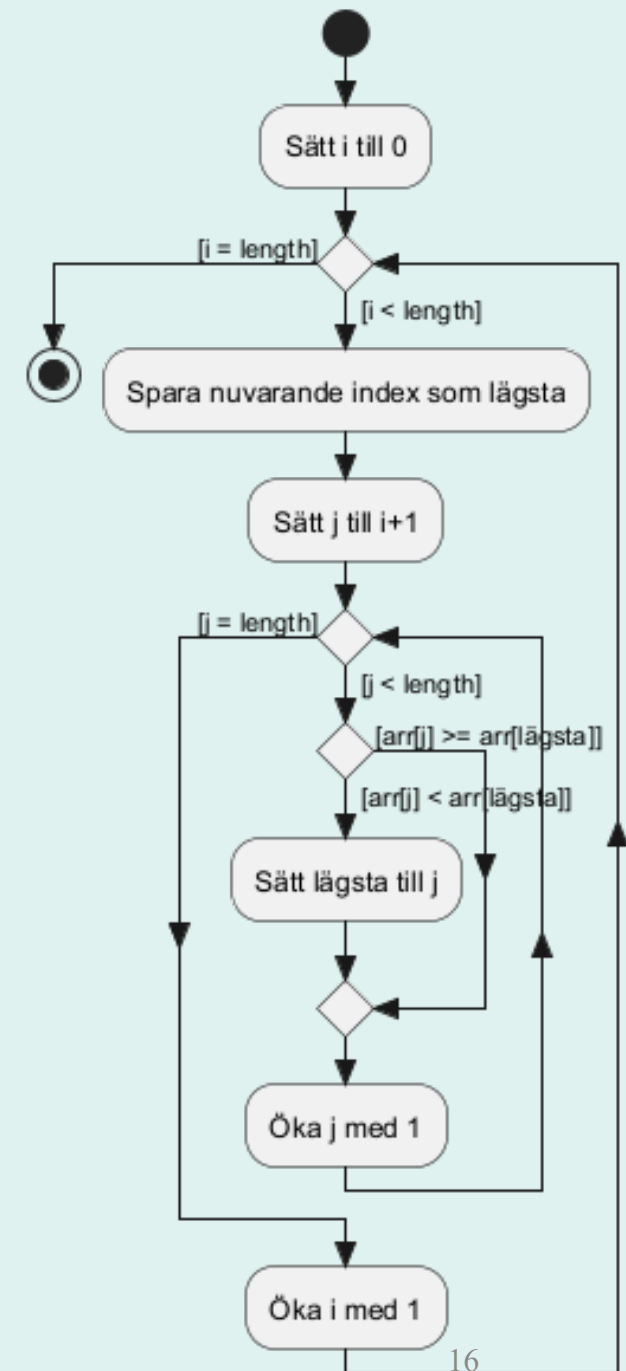
Selection Sort

```
for (int i = 0; i < arr.length; i++){  
    int smallest = i;  
    for (int j = i+1; j < arr.length; j++){  
        if (arr[j] < arr[smallest]){  
            smallest = j;  
        }  
    }  
    int temp = arr[i];  
    arr[i] = arr[smallest];  
    arr[smallest] = temp;  
}
```



Selection sort: Diagram

- Det här diagrammet är väldigt kod-nära
- Mitt verktyg för aktivitetsdiagram tillåter inte diagram som är horisontella...
- Annotera alla pilar ut från en beslutsnod.

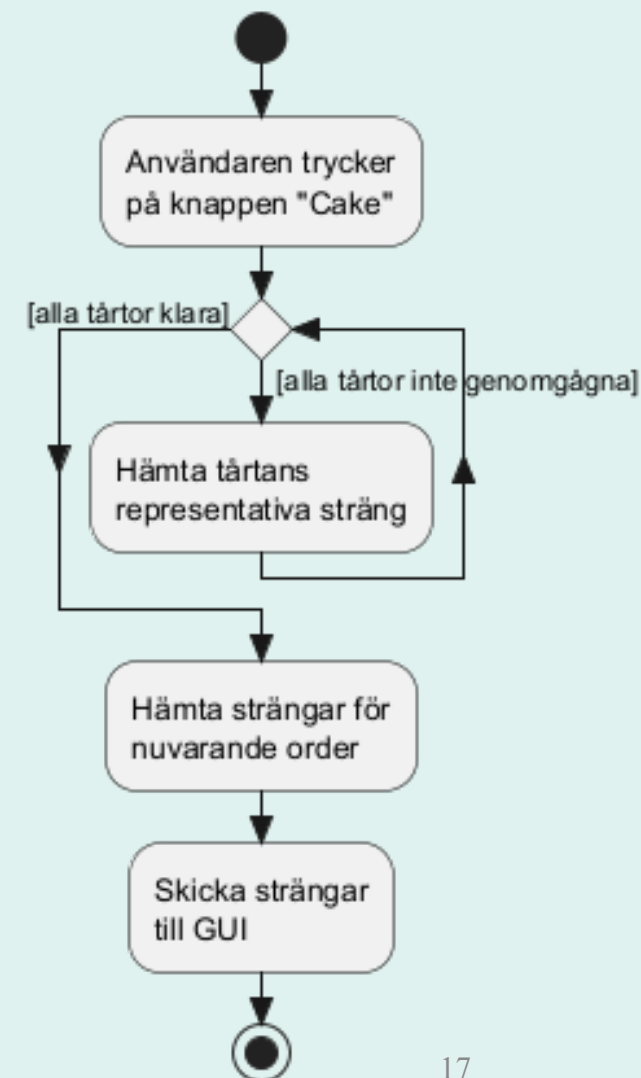


DA339A U3: Tryck på knappen "Cake"

```
String[] cakeStrings = new String[cakes.length];  
for (int i = 0; i < cakes.length; i++){  
    cakeStrings[i] = cakes[i].toString();  
}
```

```
String[] orderStrings = order.getItemStrings();  
int cost = order.getOrderPrice();
```

```
view.updateLeftPanel(cakeStrings);  
view.updateRightPanel(orderStrings);  
view.updatePriceLabel("Total: " + cost + " kr");
```

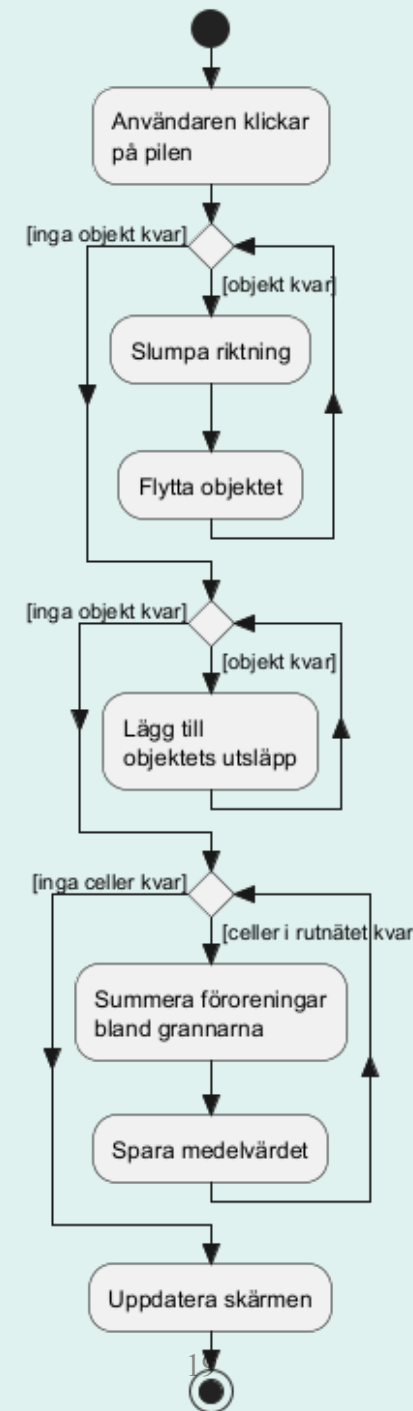
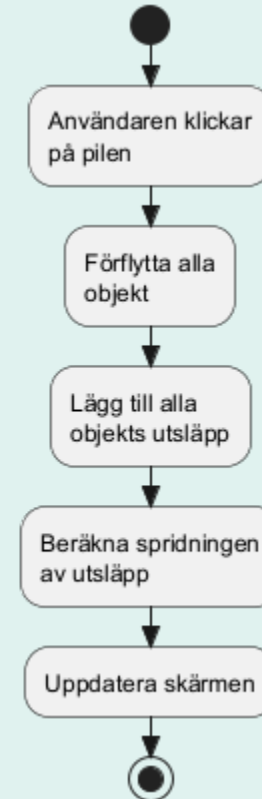


Hur formulerar man ett villkor?

- När man formulerar ett villkor (guard) så ska det vara
 - Tydligt
 - Koncist
 - Inkapslat med hakparenteser
- Detta är ibland svårt att få till

Assignment 1

- Här ser vi två diagram över aktiviteten att ”klicka på pilen” från A1
- Diagrammen har olika abstraktionsnivå
- Kan vi gå ännu djupare?

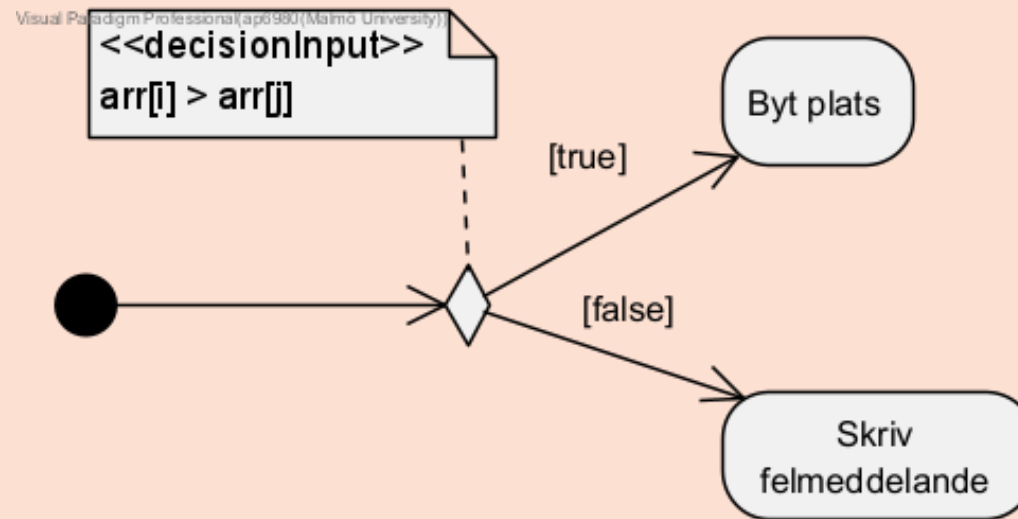




Mer komplexa diagram

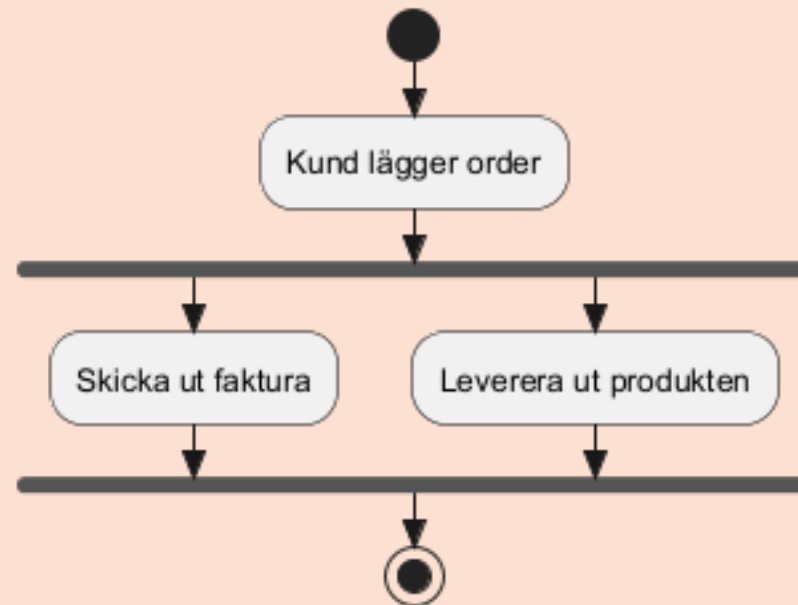
Beslutsnoder, med villkor

- Man kan visa villkoret i en beslutsnod med hjälp av en anteckning (*eng: note*)
- Anteckningen och noden ansluts med en streckad linje
- Vi markerar anteckningen med stereotypen <<decisionInput>>



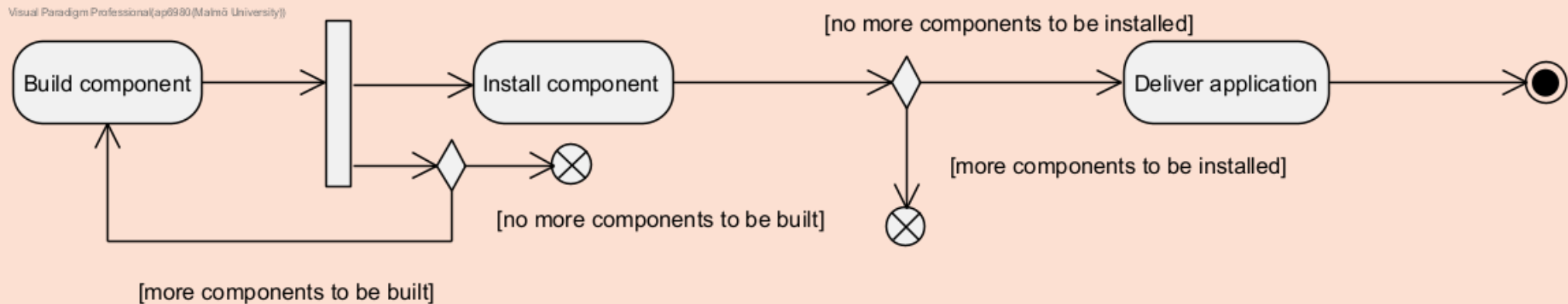
Parallella händelser

- I den här kursen kommer vi att ta upp flertrådning, där datorn kan jobba med flera instruktioner parallellt
- Detta kan visualiseras i aktivitetsdiagram med en *fork*



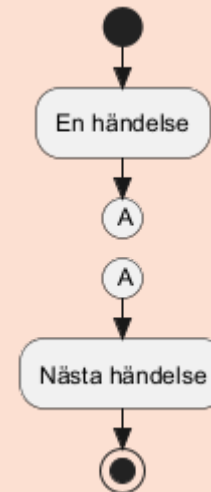
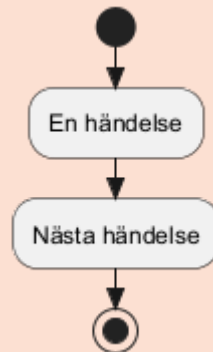
Flow final node

- En variant av *activity final node*
 - Används när en del av aktiviteten upphör, men inte hela
 - Markeras som en överkryssad cirkel
- ” A FlowFinalNode is a FinalNode that terminates a flow. All tokens accepted by a FlowFinalNode are destroyed. This has no effect on other flows in the Activity.”(UML)



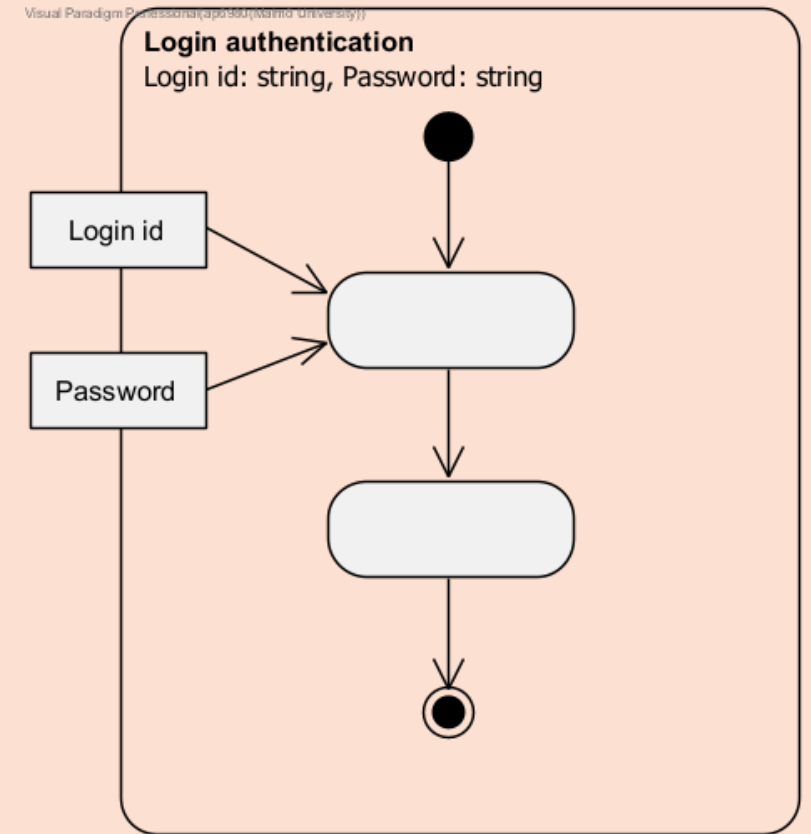
Separering, connectors

- Ibland kan man öka läsbarheten genom att använda *connectors*
- De två diagrammen nedanför är likvärdiga.



Parametrar

- Det är möjligt att lägga till parametrar till en aktivitet
- Dessa är i rektangulära lådor på kanten av en aktivitet
 - Visas även som attribut överst
- Parametrar är inputs som behövs för att aktiviteten ska utföras.



Conditionals

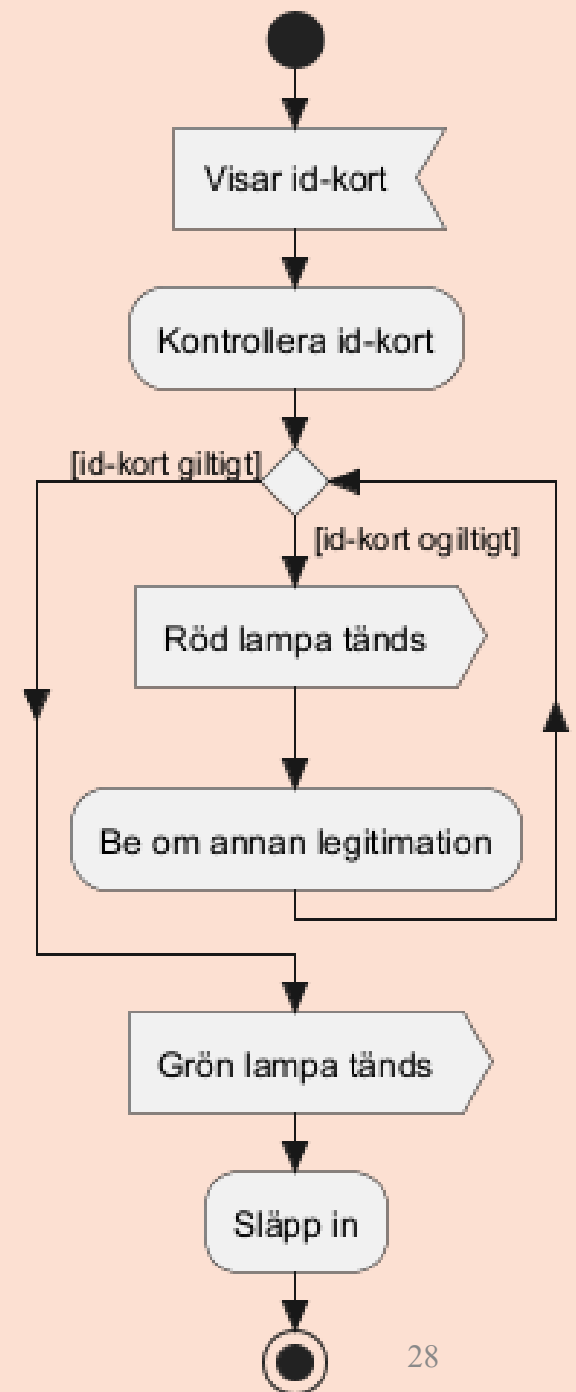
- En aktivitet, eller en handling, kan ha krav för att få påbörjas, och krav för att få avslutas
- Detta markeras med kommentarer och stereotyperna `<<precondition>>` samt `<<postcondition>>`
- Du kan se ett exempel på en av notationssidorna

Särskilda handlingar

- I UML-standarden kan vi markera olika sorters handlingar (*actions*) med olika rutor
 - <<input>>
 - <<output>>
 - <<procedure>>
 - <<load>>
 - <<save>>
 - <<continuous>>
 - <<task>>
- I den här kursen tar vi bara upp <<input>> och <<output>>

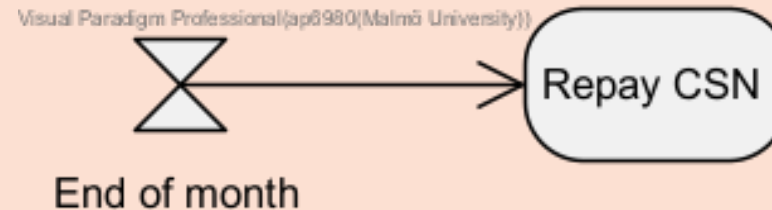
<<input>> och <<output>>

- I diagrammet till höger är ”Visar id-kort” en <<input>>
 - Detta markeras med ett ”hål” (I VisualParadigm är hålet till vänster)
- Aktiviteterna ”Röd lampa tänds” och ”Grön lampa tänds” är båda två <<output>>
 - Markeras med en pil
- I VisualParadigm matchar <<input>> och <<output>> och bildar tillsammans en hel rektangel
- Formella namn är: *Accept Event Action* och *Send Signal Action*



Timed events

- En del handlingar sker periodvis.
- Exempelvis betala CSN sker i slutet av varje månad
- Markeras med ett timglas (två trianglar som pekar på varandra)

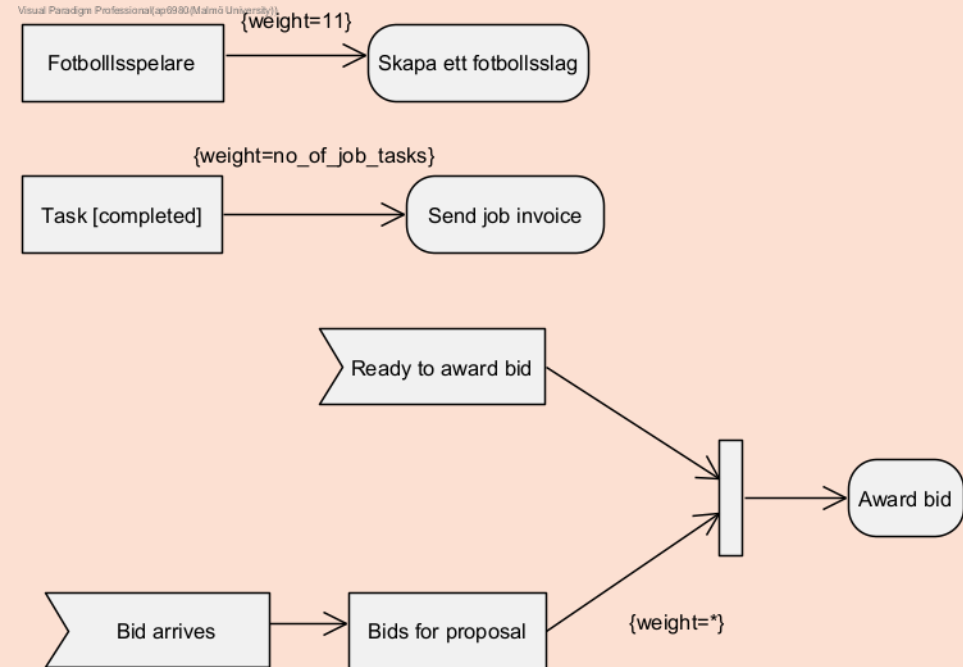


Objekt

- Om aktiviteten involverar att objekt ska flyttas mellan aktiviteter så visas objekten som rektanglar (hårda kanter)
- Objektet har sedan sitt namn innanför rutan.
- Se rutorna för parametrar, parametrarna är objekt
- Mer om objekt kommer i senare exempel

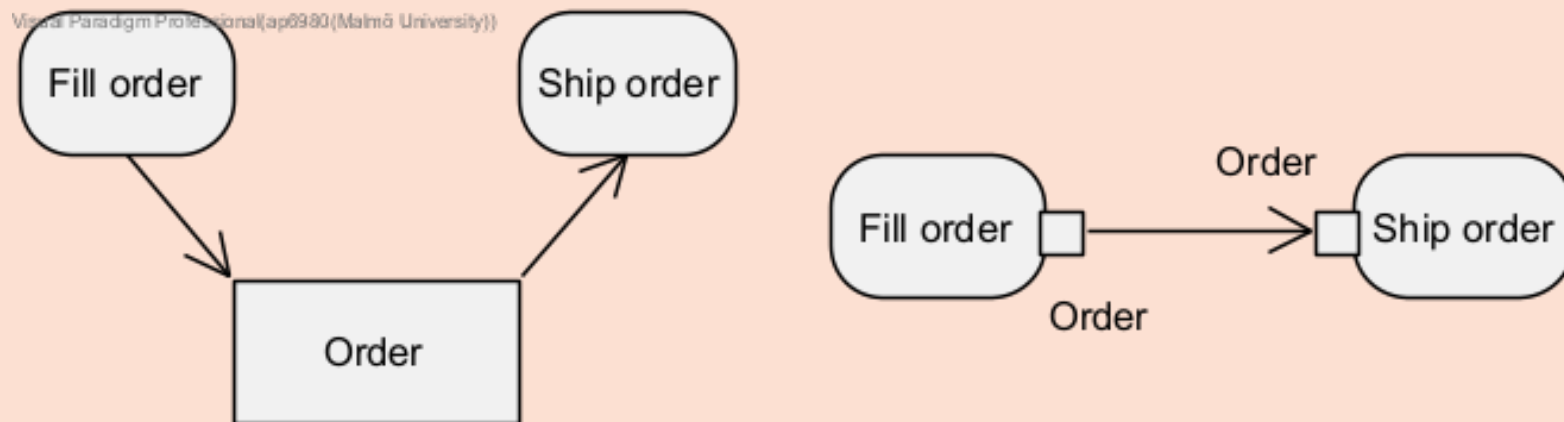
Viktade bågar

- Från objekt kan bågarna vara viktade
- Dessa fungerar som *guards* och visar hur många utav objektet som behöver finnas för att gå vidare
- Markeras med måsvingar



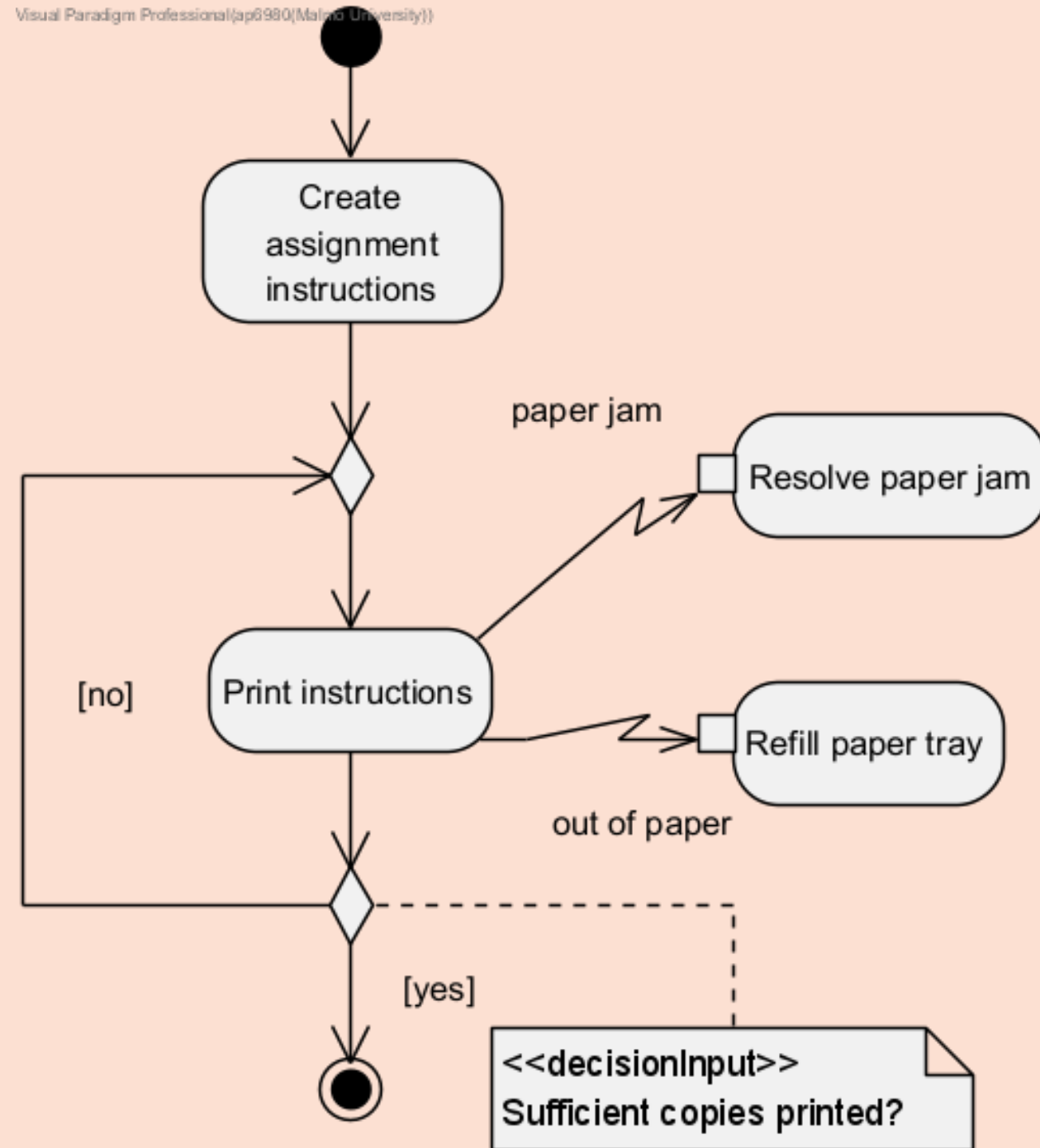
Parametrar till handlingar

- Det är möjligt att koppla objekt till handlingar
- Detta görs med en ”pin”-notation
- Man bör skriva ut datatypen på parametern
- Följande två diagram är ekvivalenta

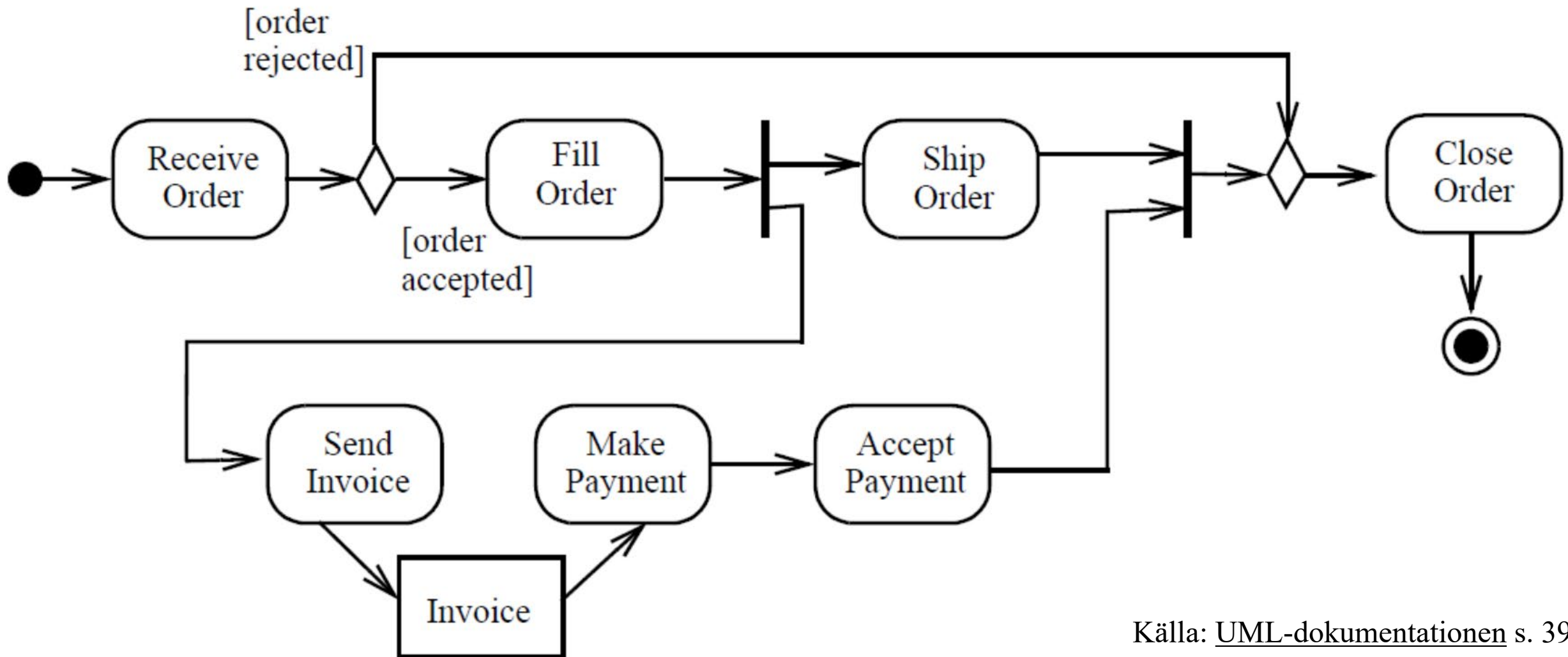


Exceptions

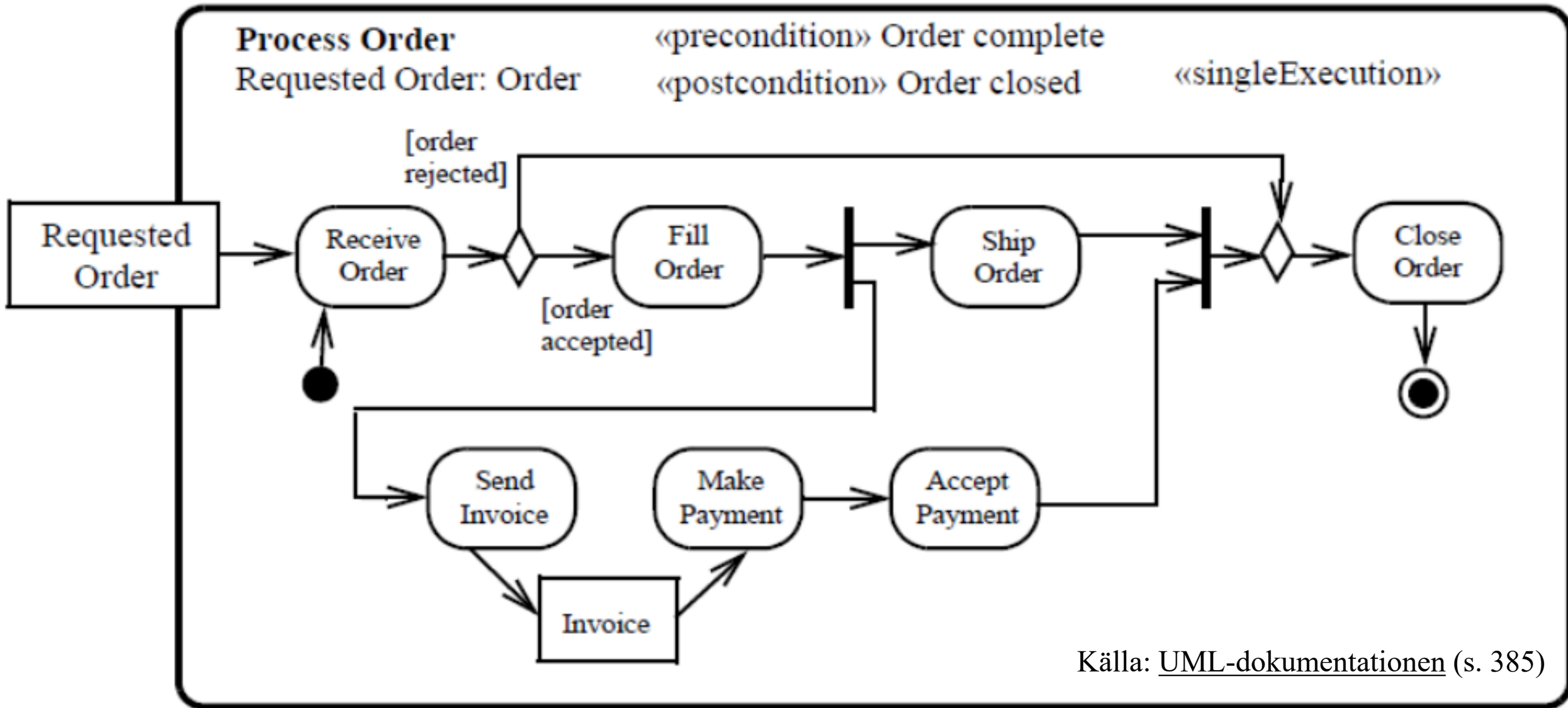
- Vi kan markera undantag med en blix
- Detta är ett undantag som kan leda till en återvändsgränd



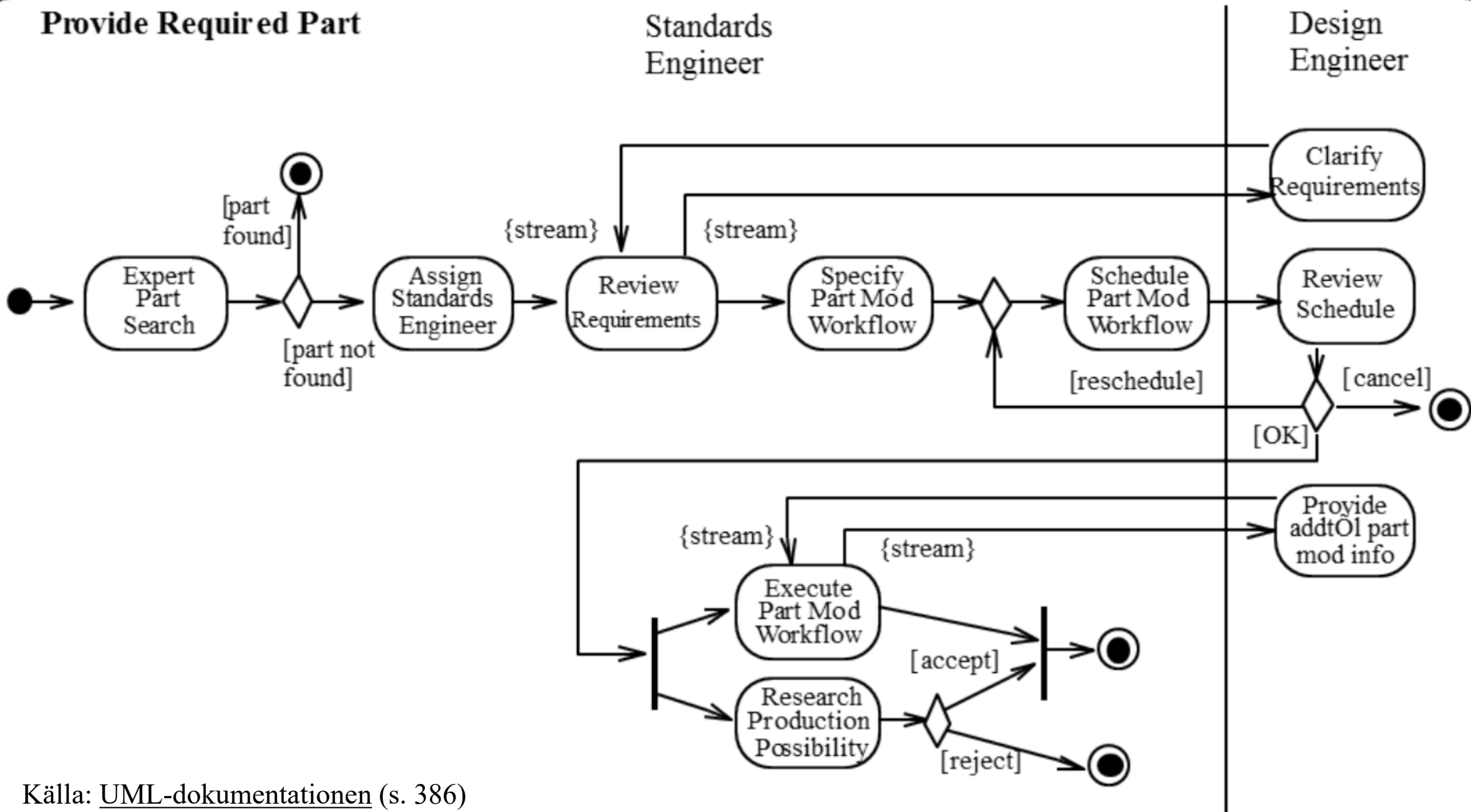
Exempel



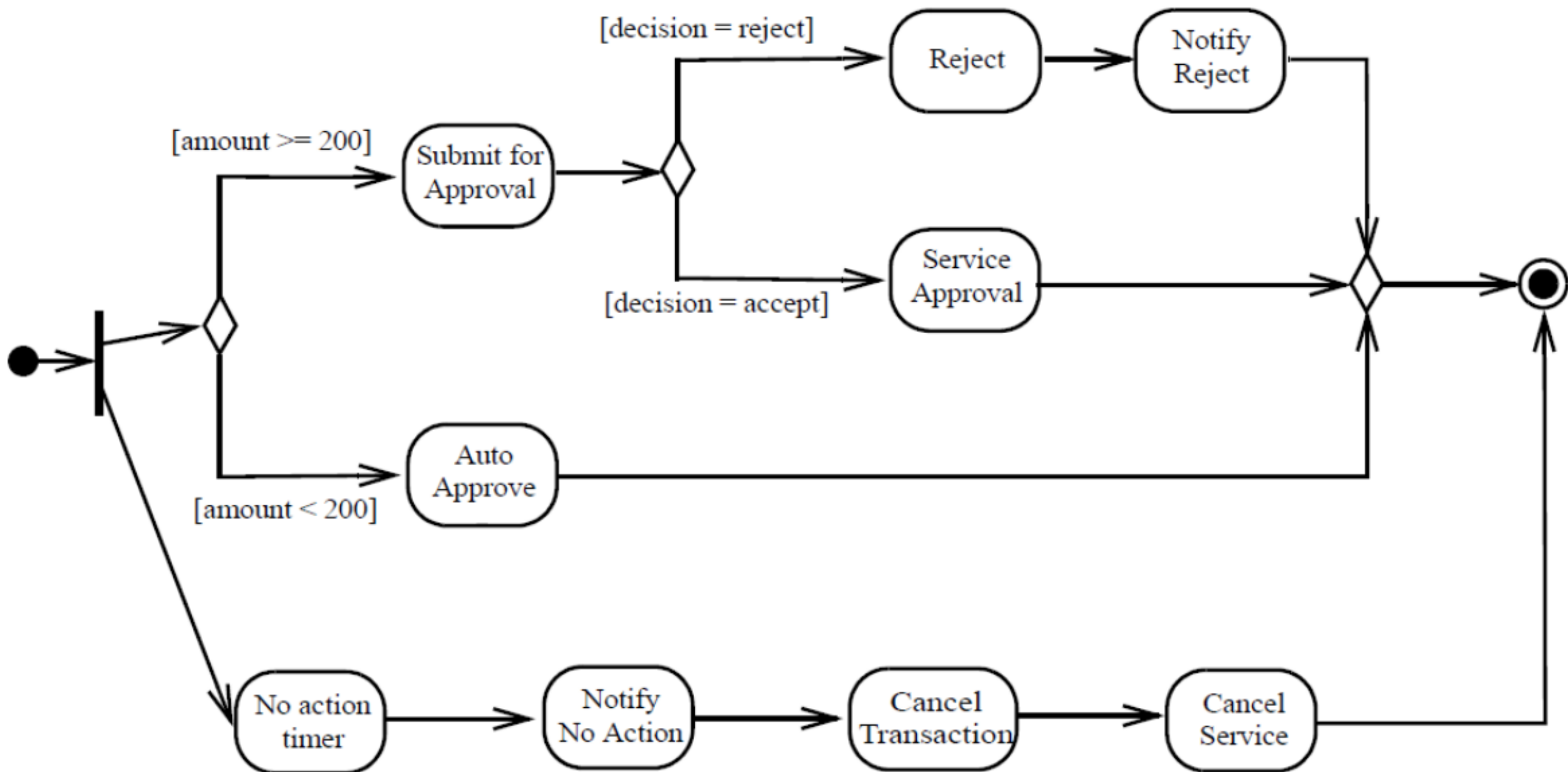
Exempel

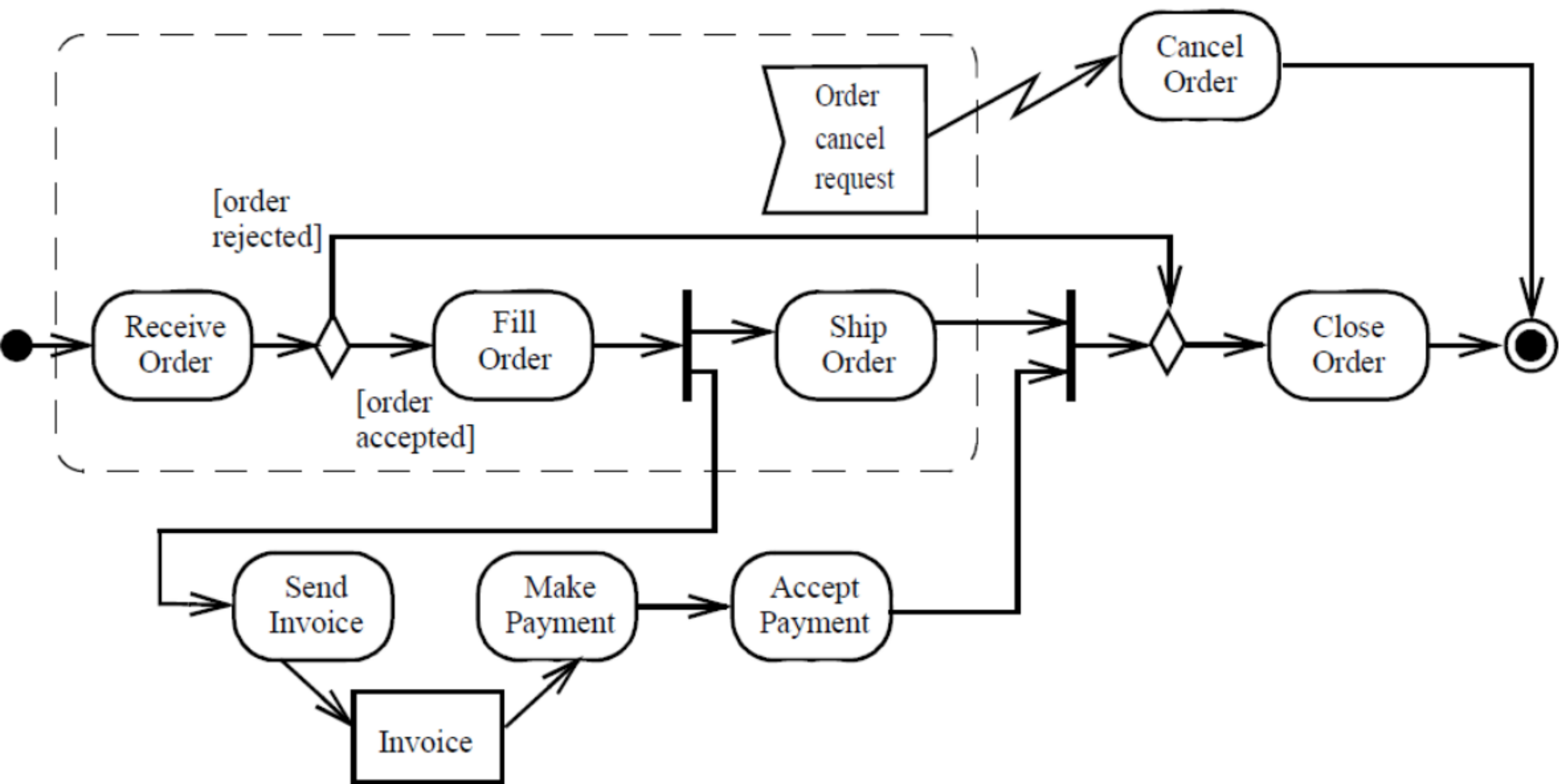


Källa: UML-dokumentationen (s. 385)



Källa: UML-dokumentationen (s. 394)

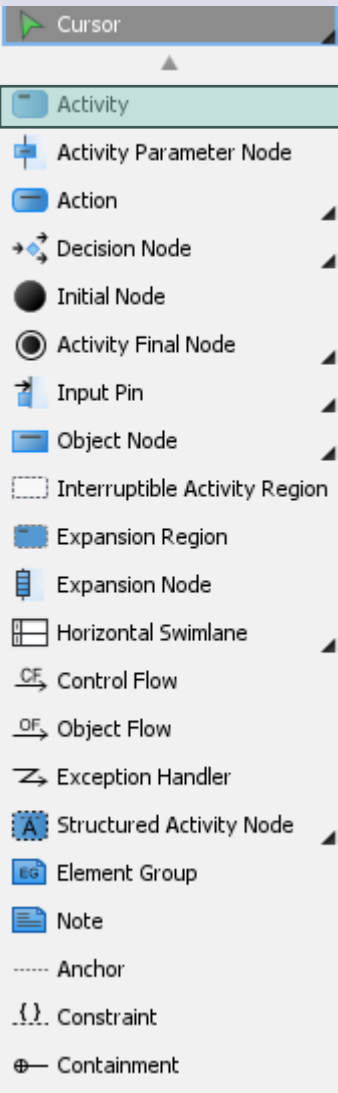






Notation och demo

Aktivitet

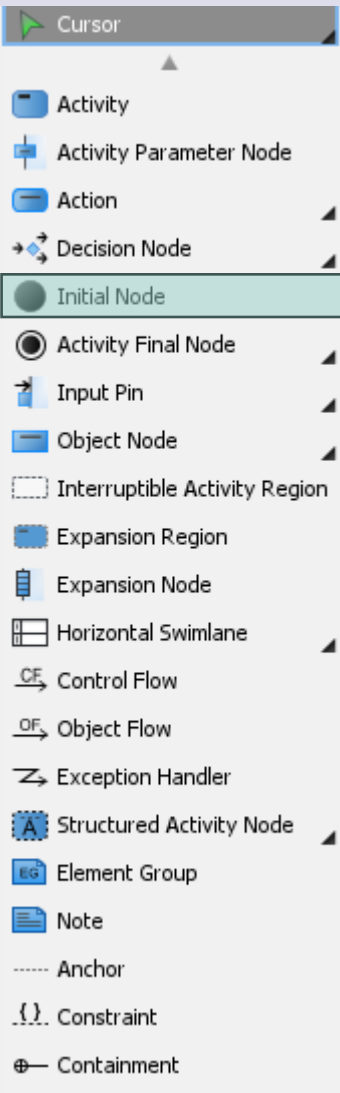


- Övergripande aktiviteten
- Behöver inte alltid visas
- Använd "Open specification" för att redigera

Aktivitetsnamn

Parametrar : typer

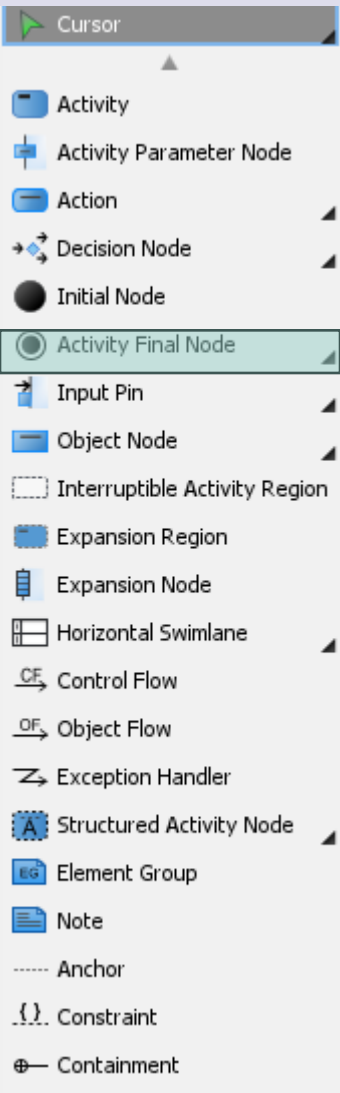
Startnod



- Punkten aktiviteten startar i
- Ifylld cirkel



Slutnod

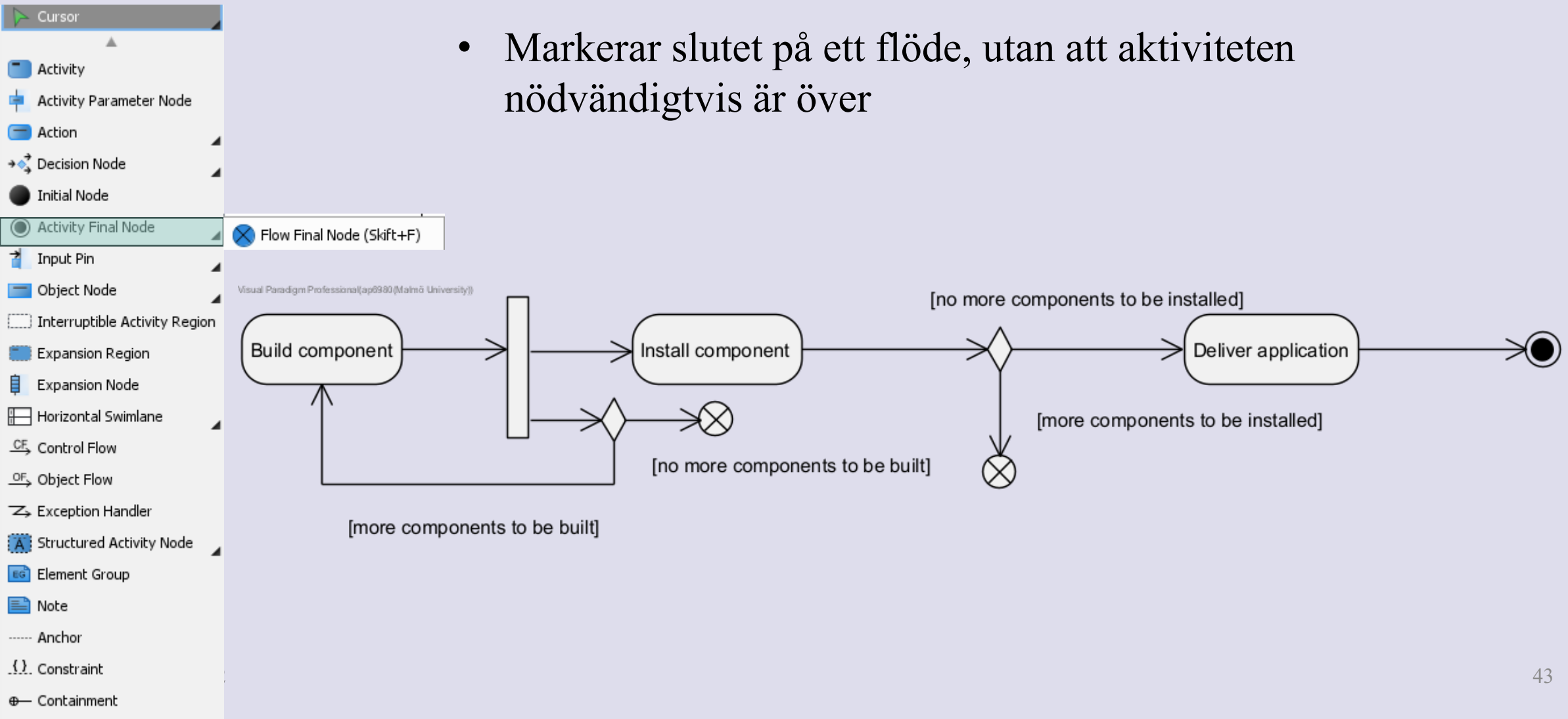


- Cirkel med en ifylld svart cirkel inuti
- Kommer vi hit är vi färdiga
- Får inte ha några pilar som går ut

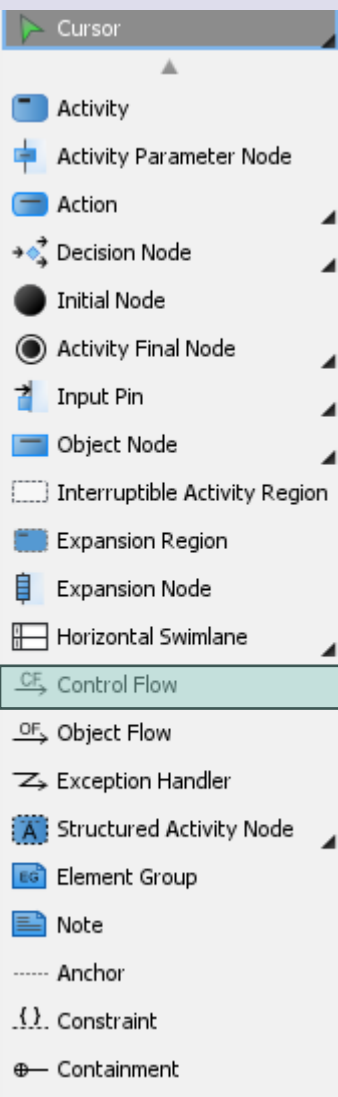


Flow final node

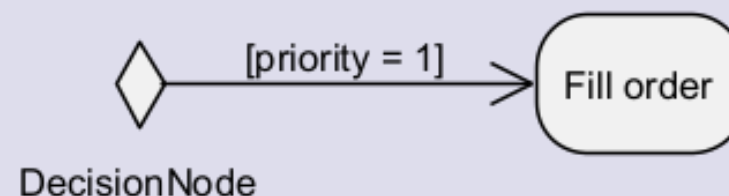
- Markerar slutet på ett flöde, utan att aktiviteten nödvändigtvis är över



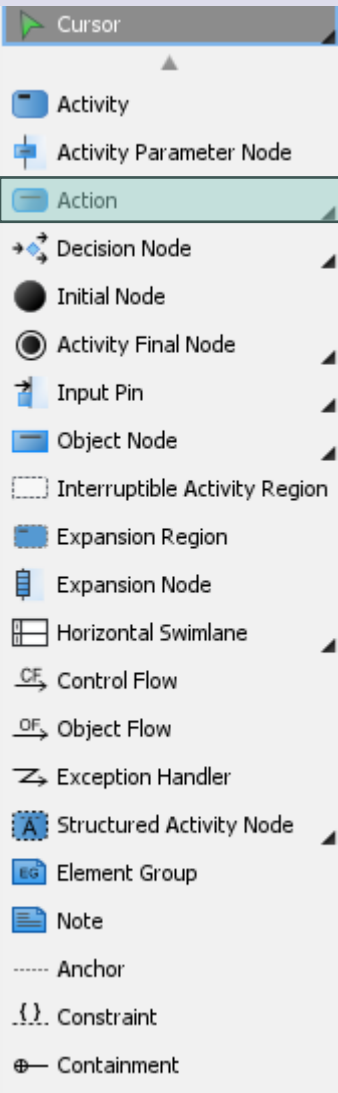
Bågar



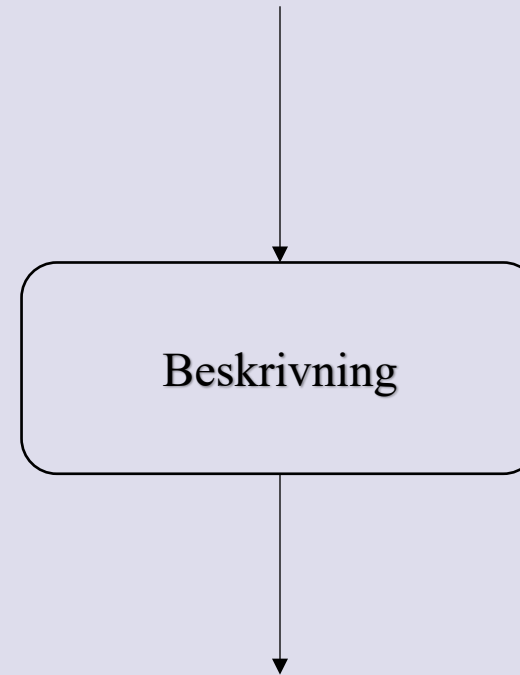
- För att koppla samman handlingar använder vi *Control Flow*
- *VisualParadigm* säger ifrån om man använder fel pil



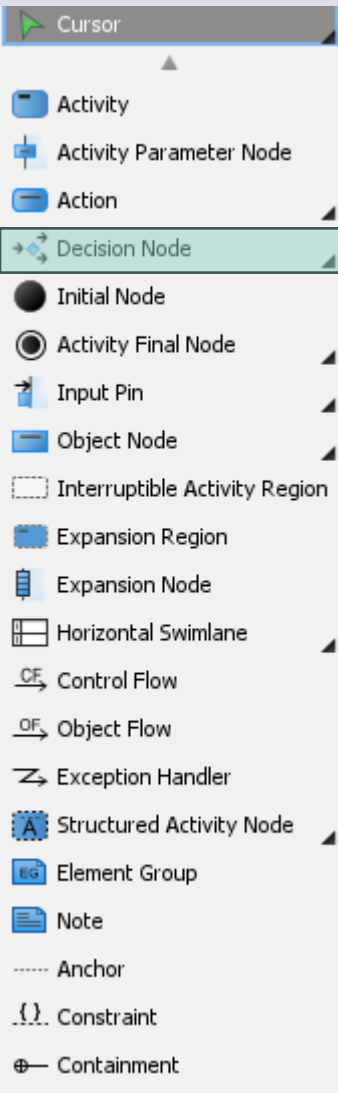
Action



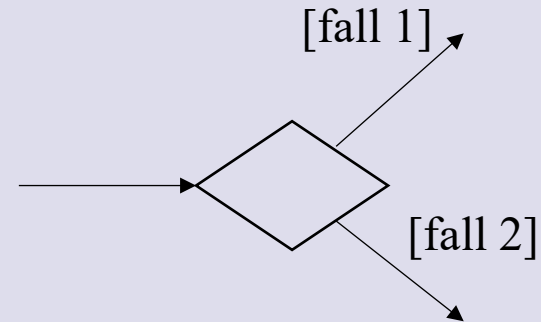
- En pil in, en pil ut som regel
- Är en "atomisk" del av en aktivitet



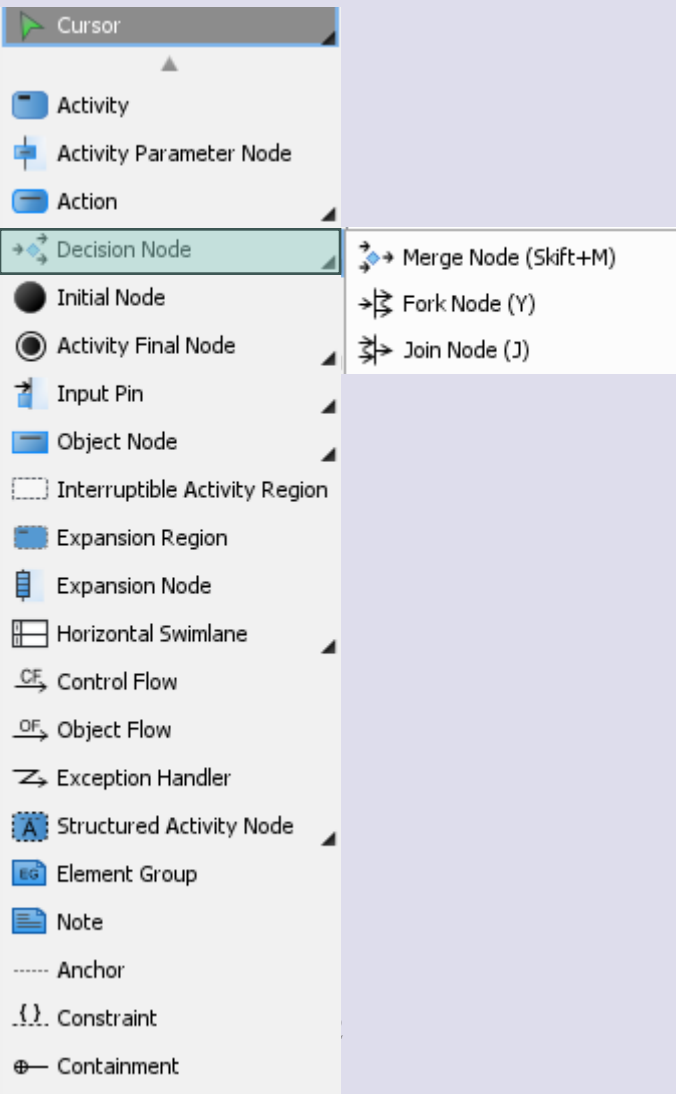
Beslutsnod



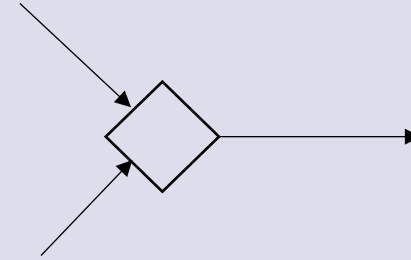
- En eller flera pilar in
- Flera pilar ut
- Ser likadan ut som en *merge*-nod
- *Guards* på ut-pilarna, markeras med hakparenteser
- Besluts-, och *merge*-noder kan kombineras



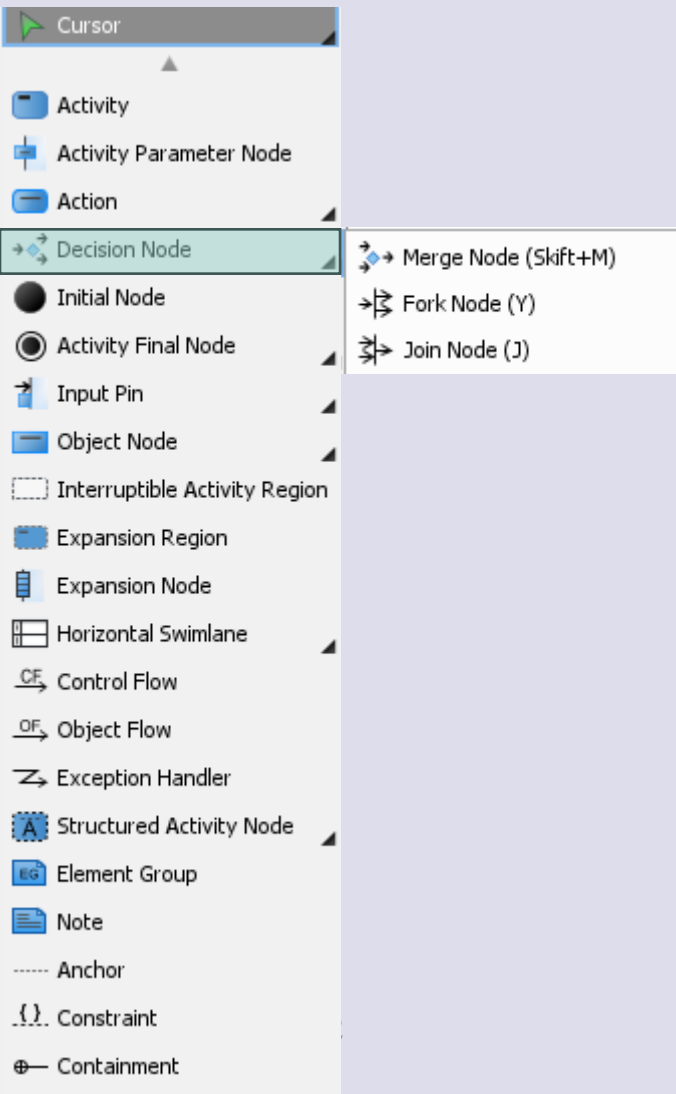
Merge-nod



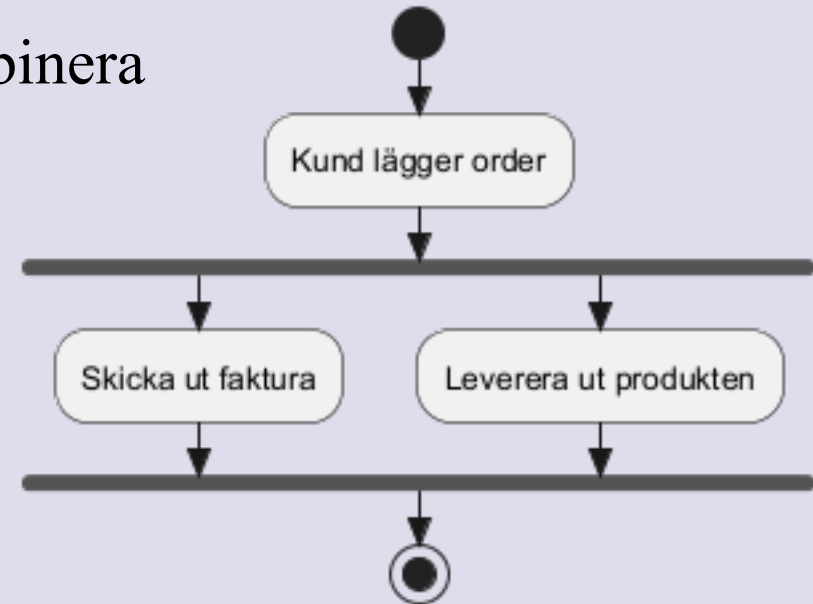
- Samlar ihop flera pilar
- En pil ut
- Ser likadan ut som en beslutsnod
- Besluts-, och *merge*-noder kan kombineras



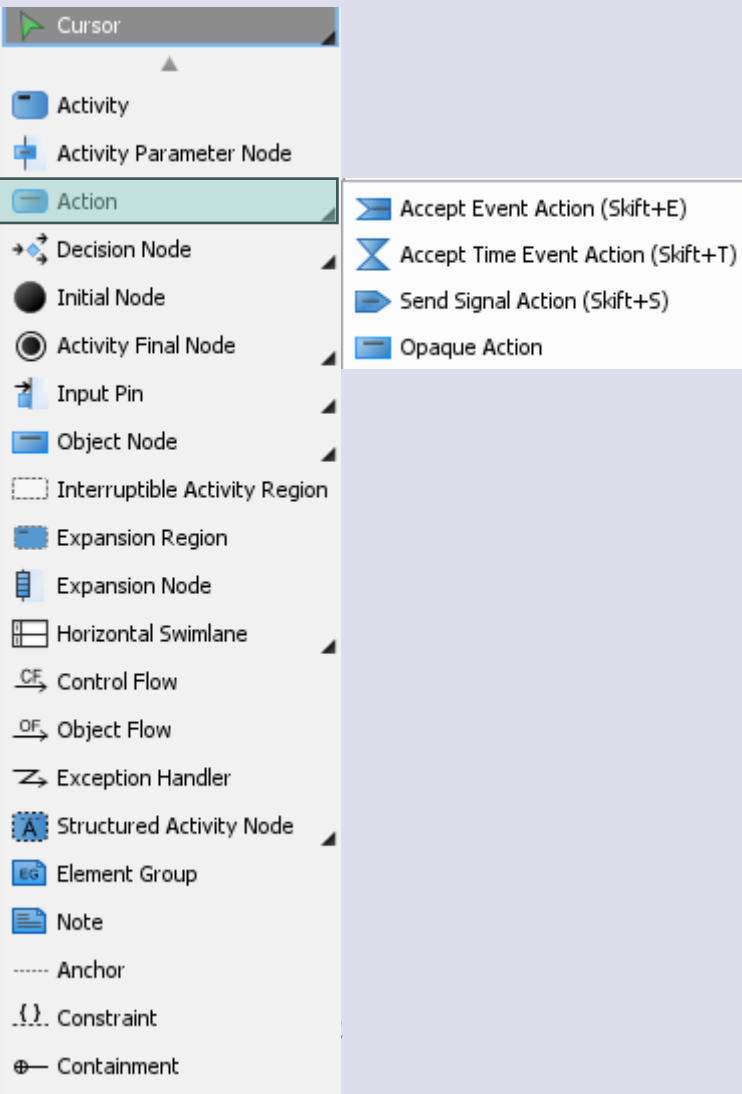
Fork- och join



- Markerar parallellisering
- *Fork* har som regel en pil in, flera ut
- *Join* har som regel flera pilar in, en pil ut
- Teoretiskt möjligt att kombinera



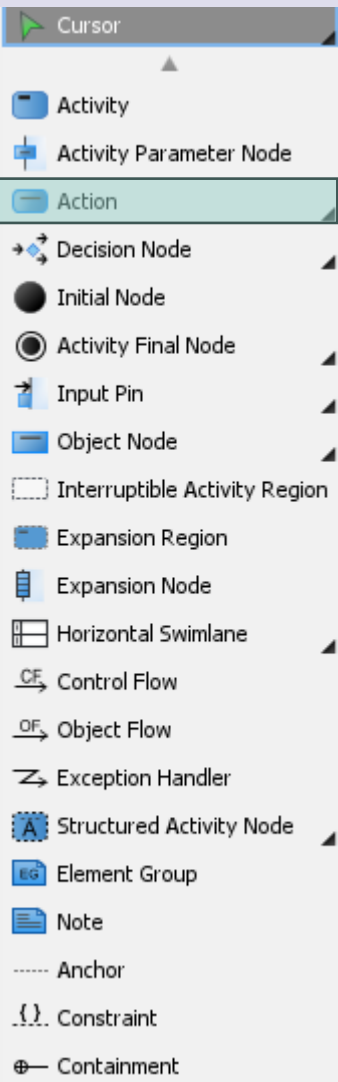
Input



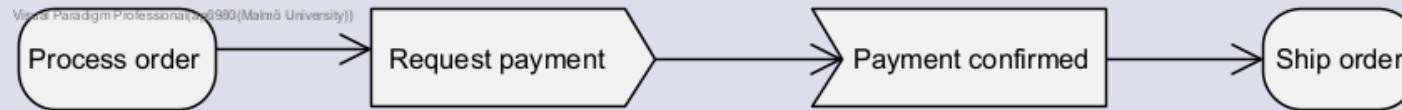
- Kallas *Accept Event Action*
- Ligger under knappen för *Action*



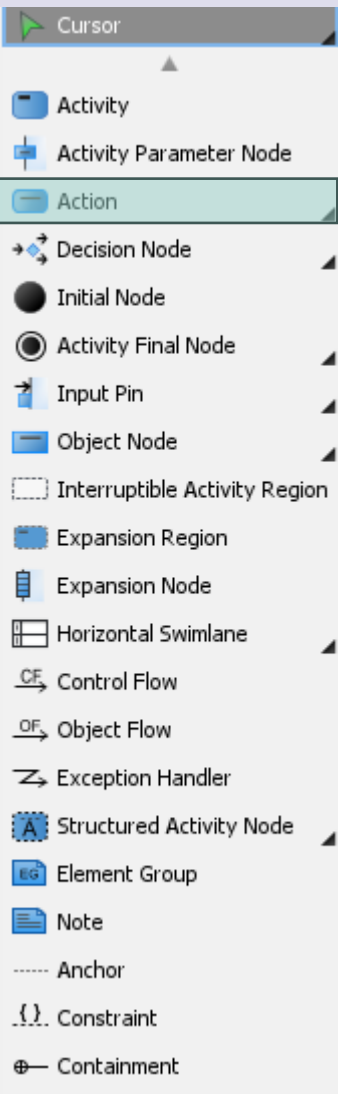
Output



- Kallas *Send Signal Action*
- Ligger under knappen för *Action*

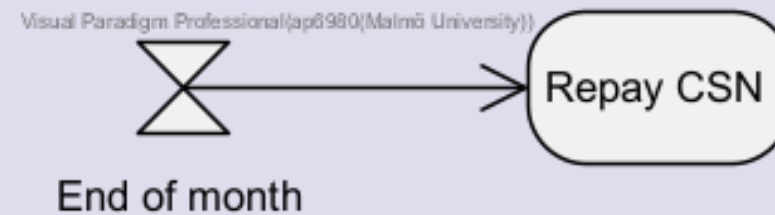


Timed event



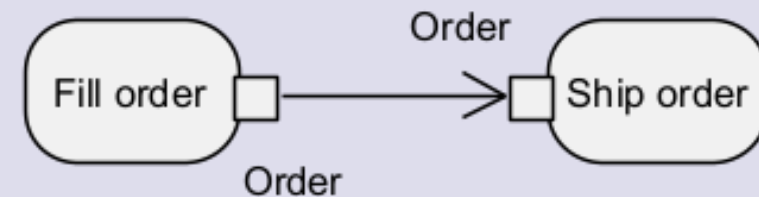
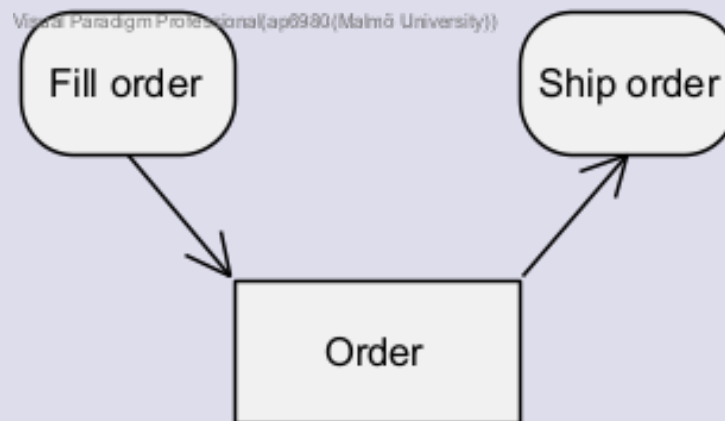
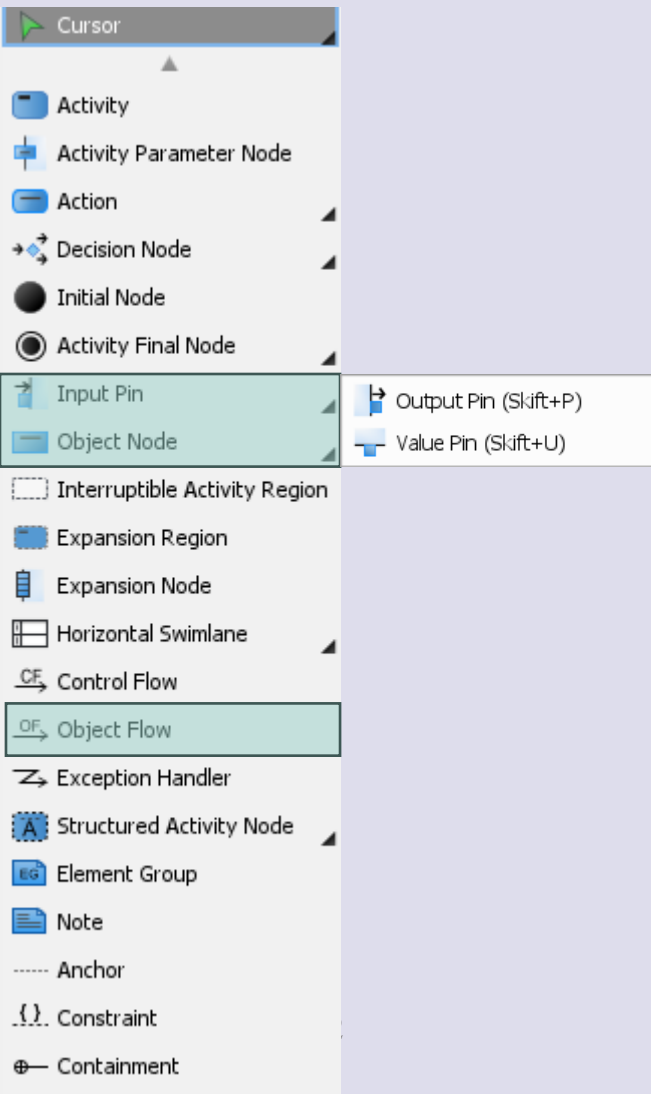
- Accept Event Action (Shift+E)
- Accept Time Event Action (Shift+T)
- Send Signal Action (Shift+S)
- Opaque Action

- Ligger under *Action*



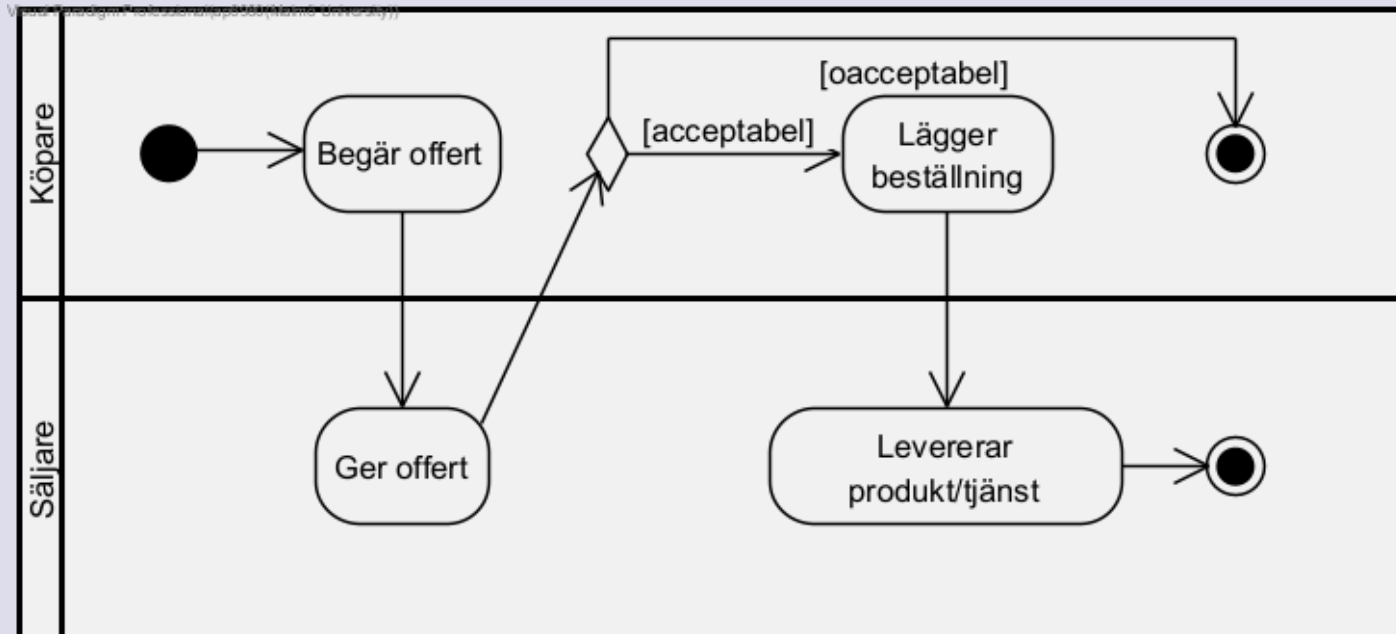
Pinned inputs/outputs

- Markerar objekt som skickas mellan handlingar
- Använder *objekt flow* pilar för att kopplas samman



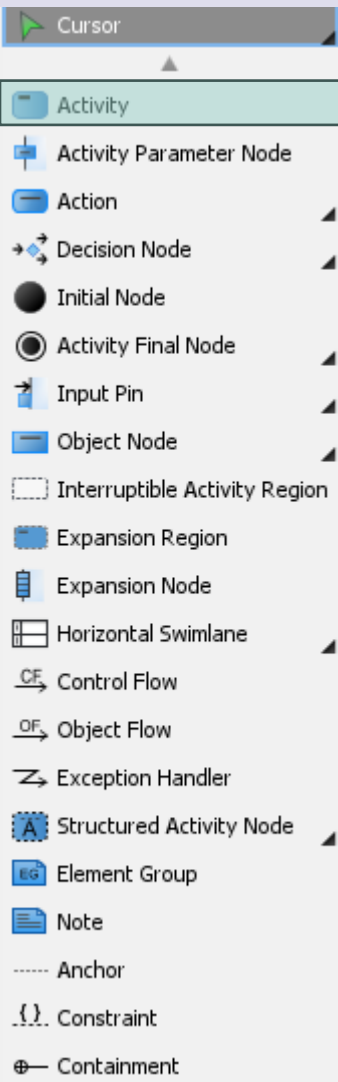
Swimlanes/partitioners

- Delar upp och visar vem som gör vad
- Finns som horisontella och vertikala
- Går att kombinera



Aktiviteter

- Mycket i en aktivitet styrs genom specifikationen



Activity Specification

General Parameters Owned Behaviors

All:

Login id

Password

Add... Remove

Selected:

Requested order

Open Specification...

Reset OK Cancel Apply Help

Activity Specification

General Parameters Owned Behaviors

Name: Process order

Visibility: <Unspecified>

Language:

Precondition:

Order complete

Postcondition:

Order closed

Body:

Description:

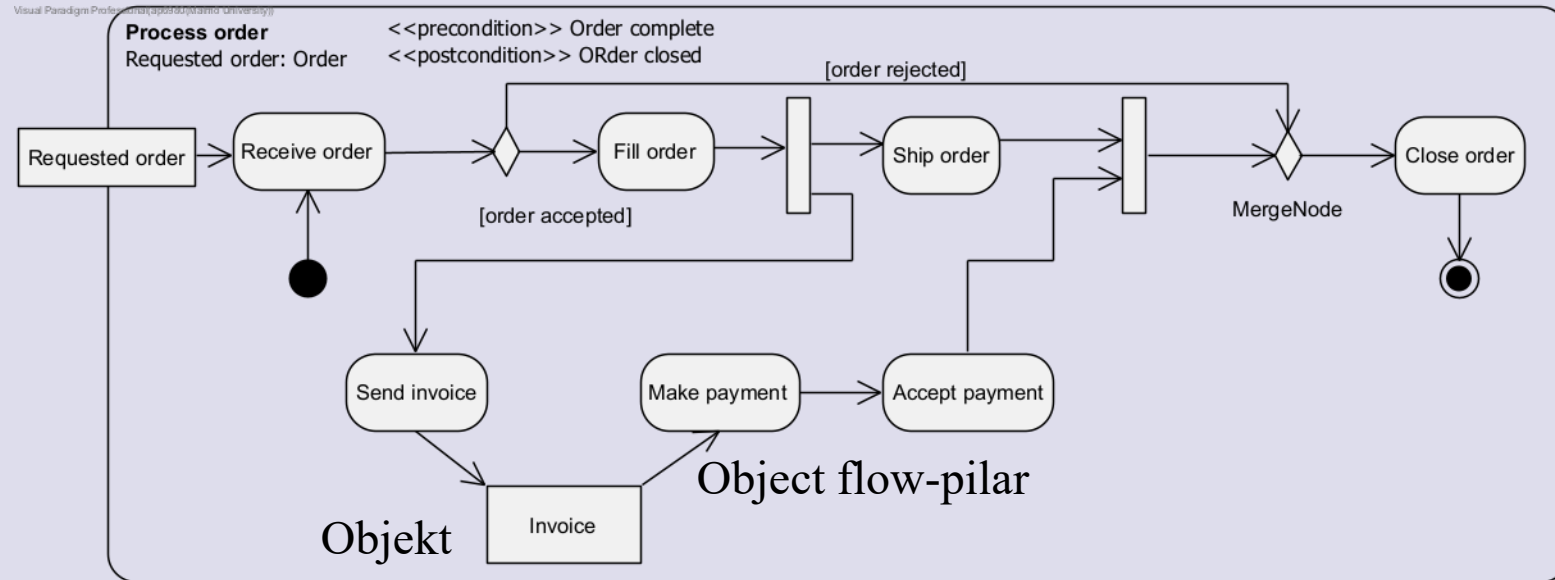
☐ Abstract ☐ Leaf ☐ Final specialization

☐ Single execution ☐ Read only ☒ Reentrant

Reset OK Cancel Apply Help

Aktivitets-parametrar objekt

- Vi lägger aktiviteter med aktivitetsknappen.
- Dessa kan omgärda handlingar
- Parametrar läggs på kanten av en aktivitet
- Objekt är rektanglar
- Har särskilda pilar





Sammanfattning

Viktiga punkter

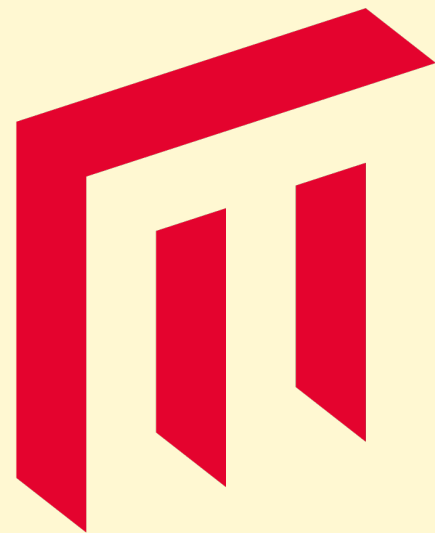
- Varje aktivitet har exakt en pil ut
- Det får inte förekomma återvändsgränder (*eng: dead ends*)
- Beslutsromber ska ha annoteringar på pilarna som går ut
 - *Merge*-romber behöver inga annoteringar
- *Swimlanes* visar vem som utför en aktivitet
- Uppgiftsbeskrivningen bestämmer abstraktionsnivån
- En *fork* visar parallellisering
- Det finns särskilda symboler för särskilda aktiviteter

Rekommenderade övningar

- Ta en vardagssituation och modellera den med ett aktivitetsdiagram
 - Gör det i olika abstraktionsnivåer
- Ta en gammal laboration eller inlämning och modellera något steg i programmet
- Jämför med en kurskamrat
 - Modellera samma aktivitet
 - Jämför era modeller
 - Hur skiljer de sig? Varför?
- Läs kurslitteraturen

Rekommenderad läsning

- Martina Seidl, *UML @ Classroom*, kap. 7, s. 141–166
- George Orwell, *1984*
- Object Management Group, *OMG Unified Modeling Language*, kap. 15 & 16, s. 373–563, <https://www.omg.org/spec/UML/2.5.1/PDF>



**MALMÖ
UNIVERSITET**

